Publicar Monky’s Store ERP en Vercel

Guía paso a paso para Windows 11 · Sin necesidad de saber programar

Objetivo: entrar a Monky’s Store desde un enlace HTTPS sin ejecutar el archivo .bat ni mantener tu laptop encendida. Importante: este proyecto aún no está publicado ni ha superado una compilación completa verificada; primero realizaremos las comprobaciones.

Qué vas a utilizar

Recurso	Para qué sirve
Carpeta del ZIP nuevo	Código revisado de Fase 4B
Node.js 22 en tu computadora	Instalar dependencias y compilar
GitHub Desktop	Subir el proyecto a un repositorio privado
Vercel	Alojar la web en internet
Supabase actual	Mantener cuentas, productos y ventas

Antes de comenzar

Haz una copia de la carpeta Fase 4B que ya utilizas y conserva un respaldo verificado de Supabase. No necesitas crear otra base de datos. Por publicar la web no debes volver a ejecutar los archivos SQL 001, 002, 003, 004 o 005.

Precaución de costos: para operar Monky’s Store comercialmente en Vercel, utiliza un plan compatible con uso empresarial. El plan Hobby está limitado a uso personal no comercial.

Parte 1 · Preparar y verificar el código

PASO 1 · Descomprime el proyecto

Descarga Monkys_Store_ERP_Fase4B_Vercel.zip. Haz clic derecho → Extraer todo. Abre la carpeta monkys-store-fase4b-vercel.

PASO 2 · Coloca tu conexión de Supabase sólo en tu computadora

Si la compilación pide variables, copia .env.local de tu carpeta Fase 4B anterior a esta carpeta nueva. No cambies la contraseña ni pongas datos reales en .env.example. El archivo .gitignore ya está preparado para no subir .env.local a GitHub.

PASO 3 · Comprueba tu Node.js

Si Node.js ya estaba instalado y funcionaba en Fase 4B, puedes continuar. Se recomienda Node.js 22. Si no lo tienes: entra a https://nodejs.org e instala la versión LTS compatible.

PASO 4 · Ejecuta la comprobación de Windows

Haz doble clic en 01_REVISAR_ANTES_DE_PUBLICAR.bat. El proceso revisa archivos, ejecuta npm install y luego npm run build. Necesitas conexión a internet. Puede aparecer una ventana negra con mensajes en inglés.

> Si aparece "ERROR" o "Build failed", detente: no publiques todavía. Copia el texto del error o toma una captura sin mostrar claves. Si termina correctamente, el proyecto estará compilado en tu computadora, pero todavía faltarán pruebas en Vercel y Supabase.

PASO 5 · Conserva package-lock.json

Después de una instalación exitosa, el archivo package-lock.json debe estar en la carpeta principal. Conserva ese archivo al publicar en GitHub. Ayuda a reproducir las dependencias probadas.

Parte 2 · Crear un repositorio privado en GitHub

PASO 6 · Instala GitHub Desktop

Abre https://desktop.github.com, instala GitHub Desktop e inicia sesión en tu cuenta de GitHub. Si no tienes una, crea una desde https://github.com.

PASO 7 · Agregar la carpeta del ERP

En GitHub Desktop: File → Add local repository. Selecciona monkys-store-fase4b-vercel. Si indica que todavía no es un repositorio, usa la opción Create a repository here.

PASO 8 · Revisa los archivos antes de publicarlos

En la lista de cambios, comprueba que NO aparezcan .env.local, contraseñas, respaldos .backup, archivos .dump ni información de clientes. Es seguro que aparezca .env.example porque solo contiene ejemplos ficticios.

PASO 9 · Publica el repositorio privado

En GitHub Desktop, guarda el primer cambio (Commit) y pulsa Publish repository. Mantén marcada la opción de código privado o desmarca "Keep this code private" sólo si muestra exactamente la opción contraria: el resultado final debe indicar Private.

> Seguridad: la clave pública de Supabase puede aparecer en el código cliente por diseño, pero nunca debes subir ni publicar la contraseña de PostgreSQL, service_role ni claves secretas. Los permisos RLS son imprescindibles.

Parte 3 · Crear la aplicación en Vercel

PASO 10 · Importar el repositorio

Abre https://vercel.com/new, ingresa con GitHub y selecciona Add New Project → Import sobre tu repositorio privado. Framework: Next.js. Root Directory: la carpeta principal del proyecto (no app).

PASO 11 · Configurar las variables de Supabase

Antes de pulsar Deploy, ve a Environment Variables. Agrega las variables de la siguiente tabla, copiando los valores de tu .env.local anterior o del panel de Supabase. Selecciona Production para estas variables.

Nombre de variable	Valor que debes proporcionar
NEXT_PUBLIC_SUPABASE_URL	La URL del proyecto que ya utilizas
NEXT_PUBLIC_SUPABASE_ANON_KEY	La clave publicable/anon del proyecto

Atención: NO pongas un usuario y contraseña de PostgreSQL, la cadena de conexión SQL ni una clave secreta en las variables anteriores. No configures las credenciales de producción en Preview si aún no tienes otra base de datos de pruebas.

PASO 12 · Primer despliegue

Pulsa Deploy. Si la compilación falla, abre Deployment → Build Logs y guarda el error, sin compartir credenciales. Si termina, Vercel te proporcionará una URL real terminada normalmente en .vercel.app.

Parte 4 · Configurar la dirección definitiva y el acceso

PASO 13 · Agregar la dirección real como variable

En Vercel → Settings → Environment Variables, añade NEXT_PUBLIC_SITE_URL con tu dirección exacta, por ejemplo https://mi-sistema.vercel.app. Selecciona Production. Redeploy para aplicar el cambio; las variables públicas se integran al compilar.

PASO 14 · Actualizar Supabase Authentication

En Supabase → Authentication → URL Configuration: establece la dirección HTTPS de Vercel como Site URL. En Redirect URLs, agrega https://TU_DOMINIO.vercel.app/auth/callback reemplazando TU_DOMINIO por el verdadero.

PASO 15 · Prueba tu cuenta administrativa

Abre la URL HTTPS de Vercel en una ventana privada. Debería redirigir al inicio de sesión. Ingresa con tu cuenta de administrador actual. Si aparece pendiente, revisa la fila de profiles en tu Supabase, sin crear otro usuario automáticamente.

Parte 5 · Lista de pruebas antes de usarla de verdad

- ☐ Sin iniciar sesión no se ve el panel, ventas, pedidos ni reportes.
- ☐ El administrador puede ingresar; el operador no accede a funciones reservadas.
- ☐ Crear un producto de prueba, generar un código de barras y comprobar su visualización.
- ☐ Reservar una unidad de prueba y comprobar stock físico versus disponible.
- ☐ Emitir un informe y descargar un CSV/Excel sólo con permisos autorizados.
- ☐ Confirmar que la conexión apunta a la base de datos correcta y que existe respaldo restaurable.

No cargues ni modifiques datos comerciales reales durante estas pruebas. Si ya tienes información real, idealmente ensaya en un proyecto Supabase independiente. Tras superar las pruebas, habilita producción con permisos mínimos.

Errores más comunes y cómo proceder

Qué observas	Acción recomendada
Build Failed	Abrir Build Logs. Verificar en Windows con el .bat de comprobación.
Faltan variables de Supabase	Revisar nombres exactos y configuración en Vercel; después Redeploy.
El correo vuelve a localhost	Actualizar Site URL, Redirect URLs y NEXT_PUBLIC_SITE_URL.
El usuario se registra pero no entra	Comprobar profiles y que la cuenta esté active=true.
Una pantalla da error SQL	Verificar Fase 4B en Supabase; no repetir migraciones a ciegas.
El sistema abre pero faltan exportaciones	Comprobar permisos y Runtime Logs en Vercel.

Qué comprobó esta entrega y qué no

Comprobado: estructura del proyecto, rutas principales, código de autenticación, protección básica de archivos privados, prueba local de lógica Code 128 y reglas de configuración.

No comprobado en este entorno: compilación final de Next.js con dependencias instaladas, acceso a tu Supabase, inicio de sesión en Vercel y pruebas completas de permisos RLS. Estas comprobaciones son obligatorias antes del uso comercial.

Además, Vercel no reemplaza un respaldo de Supabase. Guarda copias cifradas en un lugar independiente y verifica periódicamente una restauración en una base de pruebas.

Documentación oficial: vercel.com/docs · supabase.com/docs/guides/auth/redirect-urls